ledger-core — Architecture & Trade-offs

Yazan Tommalieh · November 2026 · Companion to the Part 1 repository

The core has one primitive: `balance(account, value_date_upto, as_of_posting_day)`, a sum over an append-only log where every record carries the day it was posted and the day it is effective. There is no running balance. Hold state, the once-per-day fee check and the interest schedule are all queries over that log. That is why it is correct under backdated entries, and why it is slow.

---

1. Append-only at scale

What breaks first: the end-of-day close. Every close re-checks every past day for every account, because a backdated entry can make an already-closed day negative. Each check scans the whole log. One close costs about *days × accounts × entries*, and entries grow with *days × accounts*: quadratic in both. At 100× accounts it does roughly 10,000× the work, and the close is the one job with a deadline. Nothing goes wrong; it just stops finishing on time.

Where state grows without bound. The three logs (entries, authorizations, errors) grow by design: they are the audit record. The set of seen event ids grows by accident. Every lookup is a linear scan, and one global sequence number couples all accounts, making the engine a single writer.

Cheapest fix: close the period. Pick a horizon *H*, the last closed business day. Refuse any entry value-dated before *H* — one comparison in `append_entry`. At each close, write one snapshot row per account: balance, holds and fee flags as of *H*. Then every balance is snapshot plus a scan of entries after *H*; the close re-checks only the days since *H*; the id set only needs ids after *H*. Audit is unchanged because the snapshot is derived from the log. It bounds the problem instead of speeding up the scan. Left for later: the log stays unbounded (five-year retention), and parallel processing needs a sequence per account. An index comes after those.

---

2. Value-dated entries in production

A value-dated entry rewrites history. In Part 1 one backdated debit restated three closed days, produced three fees in one close and declined an authorization; its reversal restated the days again and left the fees standing. In a UAE-licensed bank each effect has an owner.

- **Customers and complaints.** Issued statements become wrong, and a fee appears for a day the customer saw as positive. CBUAE Consumer Protection Standards (Circular 8/2020, Art. 8): acknowledge a complaint within 2 business days, answer within 30, keep the file five years. Re-assessed fees will be the main source of those complaints.
- **Fees.** Overdraft charges sit under the CBUAE retail fee caps (Regulations 29/2011, Annexure 2, amended 2018); a new fee or a change above 5% needs prior CBUAE approval. An engine that turns one backdated debit into three charges is a fee behaviour compliance must approve and the customer agreement must describe.
- **Finance.** Interest is recomputed from restated history, so a backdated entry across a month-end changes reported income; the general ledger needs reversing journals the core does not produce.
- **Audit and AML.** Every record has both dates, so “what did we know on Day 5” is a query, and the log meets the five-year retention in Federal Decree-Law No. 10 of 2025. It does not record who backdated an entry or why.
- **Context.** Under EU PSD2 Art. 87(3) a debit cannot be value-dated earlier than the debit itself, so the Part 1 backdated debit would be unlawful there as a customer payment. In a UAE bank, legitimate backdating is bank-side only: scheme clearing (T+1/T+2, carrying the transaction date), corrections and adjustments.

The one control before go-live: a backdating horizon with maker-checker. The core refuses entries value-dated before the last closed day. Anything that must land behind it goes into an adjustments queue; a second person approves it with a reason code, and the approval is appended to the log. This bounds how much history can be rewritten, creates the audit trail an examiner asks for first, and is the condition under which automatic fee re-assessment is safe. The second control — linking each fee to the entry that caused it, so reversing the cause reverses the fee — is the failing test in Part 1 and item 4 below.

---

3. Authorization lifecycle

In the model an authorization is a decision record whose state (declined, active, settled) is derived from whether a settlement references it. The only ending implemented in full is a matching settlement. The others:

- **Declined at request.** Case: not enough available funds (in practice also fraud rules). Mandate: final, with a reason code; a later backdated credit does not re-open it.

- **Duplicate request.** Case: network retry. Mandate: return the original decision, no second hold. (The model records an error instead.)
- **Never settled.** Case: abandoned pre-authorization — hotel, car rental, fuel, a cancelled order. Model: the hold lives forever. Mandate: expire per scheme rules — Visa 7 days (31 for lodging, car rental, cruise), Mastercard 7 (30 for pre-authorizations); release the hold, record the expiry.
- **Reversed by the merchant.** Case: order cancelled before capture. Model: no such event. Mandate: a new event; full reversal voids the hold, partial reduces it; only the merchant can originate it.
- **Late presentment or no authorization.** Case: stand-in processing while the issuer was down, offline chip approval, force-post. Model: rejected (Part 1 criterion 4). Mandate: post it, flagged, so the ledger matches money the scheme has already settled, and send it to disputes; the remedy is a chargeback, not refusal.
- **Settled for a different amount.** Case: final bill below the estimate; tips above it. Model: a partial settlement releases the whole hold; over-settlement posts with a warning. Mandate: partial releases the remainder unless multi-capture was flagged; over-capture within the scheme's tolerance for that merchant type.
- **Settlement later reversed.** Case: refund or chargeback. Model and mandate agree: funds return, the hold is not reinstated; a refund is a new credit and a chargeback its own workflow.
- **Account frozen or closed.** Not modelled. Mandate: a freeze blocks new authorizations but keeps existing holds; closure releases them and routes late clearings to disputes.
- **Incremental authorization.** Not modelled (same id is treated as a duplicate). Mandate: amend the existing hold and reset its expiry.

4. What I cut and why

1. **In-memory lists, linear scans, one global sequence** — persistence was out of scope. Risk: quadratic close; single writer; a crash loses the idempotency set, so a replay can double-post.
2. **Single-sided entries, no double-entry, no transfers** — the stream has none. Risk: no debits-equal-credits check; nothing reconciles to the general ledger.
3. **No hold expiry, merchant reversal, incremental authorization or multi-capture** — none in the stream. Risk: holds lock funds forever; a merchant void, the most common real ending, cannot be represented.
4. **No link from a fee to the entry that caused it, no fee reversal** — the spec says when to assess, never when to unwind; this is the deliberately failing test. Risk: a reversed bank error leaves the customer paying. Fix: a trigger reference on each derived entry and a compensating fee reversal.
5. **Approvals never revisited after a backdated debit** — card networks work this way. Risk: the exposure sits with the bank; nothing flags “approved on a balance later restated”.
6. **Debits never declined, no overdraft limit, flat fee with no cap** — the spec gives none. Risk: unlimited overdraft and fee stacking; production needs a limit per account and a cap per cycle.
7. **Simple daily interest, positive balances only, one capitalisation; no debit interest, tiers, or a profit model for an Islamic window** — spec wording. Risk: wrong for any real product; a restated accrual across a reporting period has no journal.
8. **Integer days, no calendar** — the brief's days are abstract. Risk: weekends (Saturday–Sunday here), holidays, cut-offs and Gulf-time day boundaries are undefined.
9. **Future value dates refused** — keeps “balance as of day d” unambiguous. Risk: standing and scheduled payments cannot be represented.
10. **Backdating unbounded** — the exercise depends on it. Risk: full-history restatement; covered by the horizon in section 2.
11. **Structural faults stop the replay** (precision, unknown account, duplicate id, bad amount) — right for a test harness. Risk: one bad message halts production; needs a dead-letter queue.
12. **Rejections are only logged** — the brief asks for errors to be printed. Risk: a real force-post never reaches disputes; settled money goes unaccounted.
13. **No operator, channel, reason code or external reference on entries** — the stream carries none. Risk: audit knows what and when, not who or why; no reconciliation to scheme files.
14. **Only an AED fee configured** — the spec gives one. Risk: an overdrawn BHD account produces an error, not a charge; needs per-currency configuration.
15. **HALF_UP, rounded daily then summed** — spec wording. Risk: must match the customer agreement (the difference is fils).
16. **No account lifecycle** — out of scope. Risk: freeze and closure behaviour for holds is undefined.

In one line: the core is a query over an immutable log — right and slow. Production keeps the log and adds a horizon, links on derived entries, and the authorization events the card world actually sends.

References: CBUAE Consumer Protection Standards (Circular 8/2020) Art. 8 · CBUAE Regulations 29/2011 Annexure 2 (2018) · Federal Decree-Law No. 10 of 2025 · PSD2 Art. 87 · Visa/Mastercard hold timeframes. Drafted with Claude, as disclosed in the Part 1 WORKLOG.